Customer Churn Prediction & Agentic Retention Strategy

Project 5 — Milestone 2: End-Semester Submission

GenAI Course — Agentic AI & RAG Pipeline Report

Dataset	IBM Telco Customer Churn (Kaggle)
Agent Framework	LangGraph (5-node StateGraph)
LLM	Groq `llama-3.3-70b-versatile` (free tier)
RAG Store	ChromaDB + `all-MiniLM-L6-v2`
UI	Gradio (two-tab interface)
Hosting	Hugging Face Spaces
Phase	Milestone 2 — Agentic AI

Team Members

Aaloke Eppalapalli – 2401010005

Arina Ali – 2401020114

Hemanth Tenneti – 2401010186

Shrihari K N – 2401010448

April 19, 2026

Contents

1. Abstract

This report documents the design and implementation of Milestone 2 of Project 5: the extension of a classical machine learning churn prediction system into a fully agentic AI retention strategy assistant. Building upon the Logistic Regression pipeline established in Milestone 1, this phase introduces a five-node LangGraph state machine that autonomously reasons about customer churn risk, retrieves evidence-based retention strategies from a curated knowledge base using Retrieval-Augmented Generation (RAG), and generates structured, personalised retention reports through a free-tier large language model.

The agentic system integrates ChromaDB as a persistent vector store, the `all-MiniLM-L6-v2` Sentence Transformer for local embedding generation, and Groq's `llama-3.3-70b-versatile` as the reasoning LLM. The complete pipeline is deployed publicly on Hugging Face Spaces and operates within free-tier API constraints. The system produces structured outputs comprising a risk summary, identified risk factors, a plain-English explanation, three personalised retention recommendations grounded in retrieved knowledge, and a single executive summary sentence.

2. Introduction

2.1. Business Context and Motivation

Milestone 1 demonstrated that historical customer behavioural data carries strong predictive signal for churn, with Logistic Regression achieving an F1-Score of 0.6092 and a ROC-AUC of 0.8416. However, prediction alone is insufficient for operational retention. A customer service team receiving a list of high-risk customers still faces the challenge of deciding *what to do* — which intervention to offer, how to frame it, and what retention strategies are most applicable to a given customer's specific risk profile.

Milestone 2 addresses this gap by extending the prediction system into an autonomous reasoning agent. Rather than requiring a human analyst to interpret model outputs and design interventions, the system reasons about risk, retrieves relevant retention strategies from a structured knowledge base, and generates a complete, actionable retention report that a customer service representative can act on directly.

2.2. Problem Statement

Given a customer's 20 demographic, service, and billing attributes, the agentic system must:

- Run the trained ML model to obtain a churn probability and risk classification.
- Identify the primary risk factors driving the prediction using deterministic rules.
- Retrieve the most relevant retention strategies from a curated knowledge base.
- Generate a plain-English explanation of why the customer is at risk.
- Produce exactly three personalised, actionable retention recommendations grounded in retrieved knowledge.
- Summarise the entire analysis in a single executive sentence.

- Expose all outputs — including retrieved source chunks — through a professional web interface.

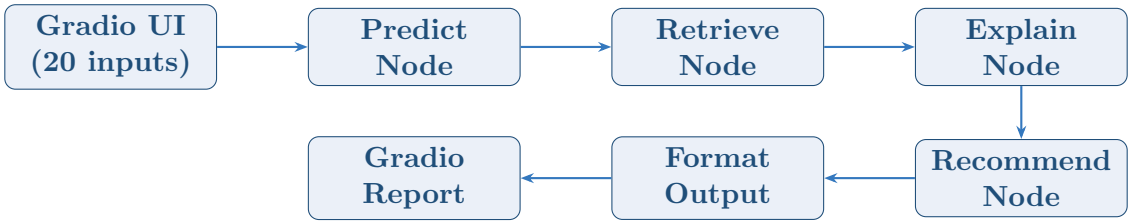
2.3. Scope of Milestone 2

This milestone is scoped exclusively to agentic and generative AI methods. The classical ML model from Milestone 1 is retained as a tool within the agent pipeline but is not retrained or modified. All LLM inference uses the Groq free tier. All embeddings are generated locally using open-source Sentence Transformers. No paid embedding APIs are used.

3. System Architecture

3.1. High-Level Overview

The Milestone 2 system is a multi-component pipeline in which the Gradio web application serves as the entry point, the LangGraph agent orchestrates the reasoning workflow, and ChromaDB provides the retrieval backbone. The overall data flow is as follows:



3.2. Repository Structure

The codebase is organised into three top-level modules, maintaining a clean separation between agent logic, RAG infrastructure, and the user interface:

```
CustomerChurnPredictor/
-- app.py                      # Gradio two-tab UI
-- agent/
   -- churn_agent.py          # LangGraph 5-node state machine
   -- tools.py                # LangChain @tool wrappers
   -- prompts.py              # LLM prompt templates
-- rag/
   -- ingest.py               # ChromaDB vector store builder
   -- retriever.py            # Query-time RAG interface
   -- knowledge_base/
       -- retention_strategies.md
-- models/
   -- model.pkl               # Serialised Logistic Regression
-- scaler.py                   # StandardScaler builder
-- requirements.txt
-- .env.example
```

4. Agentic Workflow: LangGraph State Machine

4.1. State Schema

The agent is implemented as a `StateGraph` in LangGraph. All inter-node communication is mediated through a single `AgentState` TypedDict, which carries the full context of the analysis across all five nodes:

```
class AgentState(TypedDict):
    customer_features: dict
    churn_probability: float
    churn_prediction: bool
    risk_level: str                # "Low" | "Medium" | "High"
    risk_factors: list[str]
    retrieved_context: str
    explanation: str
    recommendations: list[str]
    executive_summary: str
    error: str | None
```

This design ensures that every node has access to all upstream outputs without explicit parameter passing, and that the final state object returned to the UI contains the complete analysis record.

4.2. Node 1: Predict

The `predict` node encodes the 20 raw customer attributes into the 30-column feature vector expected by the Logistic Regression model, applies the pre-fitted `StandardScaler`, and calls `predict_proba()` to obtain the churn probability. Risk level thresholding is applied as follows:

- **High risk:** $p \geq 0.7$
- **Medium risk:** $0.4 \leq p < 0.7$
- **Low risk:** $p < 0.4$

In parallel, the `identify_top_risk_factors_tool` applies deterministic business rules to identify up to three risk factors from the customer profile — for example, month-to-month contract, short tenure (under 12 months), fibre optic subscription, electronic check payment, or high monthly charges (above $70). These factors are passed downstream to guide both retrieval and LLM generation.

4.3. Node 2: Retrieve Context

The `retrieve_context` node constructs a natural language query string from the customer's risk level and identified risk factors — for example, *"retention strategies for High risk: month-to-month contract, short tenure, fiber optic subscriber"* — and passes it to the RAG retriever. The retriever queries the ChromaDB vector store using Max Marginal Relevance (MMR) search, returning the top three chunks that are both highly relevant to the query and maximally diverse relative to one another. The retrieved chunks are concatenated and stored in the `retrieved_context` field of the state.

4.4. Node 3: Explain

The `explain` node calls the Groq LLM using the `EXPLANATION_PROMPT` template. The system prompt instructs the model to act as a senior customer success analyst and produce a 3–4 sentence plain-English explanation of why this specific customer is at churn risk, referencing their actual feature values. The prompt explicitly prohibits machine learning jargon. Both the customer profile and the retrieved retention context are provided to ground the explanation in domain knowledge.

4.5. Node 4: Recommend

The `recommend` node calls the LLM using the `RECOMMENDATION_PROMPT` template. The system prompt instructs the model to act as an expert retention specialist and produce exactly three numbered, personalised, actionable recommendations that a customer service agent could execute on a real call. Each recommendation must reference the customer's specific situation and be grounded in the retrieved retention strategies. The LLM output is parsed into a clean list by stripping numbering prefixes, and the first three items are retained.

4.6. Node 5: Format Output

The `format_output` node makes a final LLM call using the `SUMMARY_PROMPT` template, requesting a single executive summary sentence under 25 words that captures the retention action priority for this customer. This sentence is designed to appear at the top of a management-facing report, providing an immediate action headline before the detailed analysis.

4.7. Error Handling

Every node is wrapped in a `_safe()` decorator that catches all exceptions, logs them, and writes a structured error message to the `error` field of the state. This ensures that the LangGraph graph never crashes mid-execution and that the Gradio UI always receives a valid state object. If the GROQ_API_KEY is absent or invalid, the UI renders a user-friendly "Agent Unavailable" panel rather than an unhandled exception.

4.8. Singleton Agent Instance

The `ChurnRetentionAgent` is instantiated once per process via a lazy singleton pattern (`get_agent()`). This avoids recompiling the LangGraph state machine on every request, which is critical for responsiveness on the hosted Hugging Face Space.

5. Retrieval-Augmented Generation Pipeline

5.1. Knowledge Base

The RAG knowledge base is a curated markdown document (`retention_strategies.md`) containing evidence-based retention strategies organised across eight customer segments:

1. Month-to-month contract customers
2. Fibre optic internet subscribers

3. Electronic check payment customers
4. Short-tenure customers (0–12 months)
5. Senior citizen customers
6. Customers without value-add services
7. High monthly charge customers (above $70/month)
8. General retention principles and economics

Each segment contains multiple specific, actionable tactics — for example, proactive outreach at 30/60/90-day milestones for new customers, auto-pay enrolment incentives for electronic check payers, and speed upgrade promotions for fibre optic subscribers. The document also contains quantitative retention economics, including the finding that a 5% improvement in retention rates can increase profitability by 25–95% [1] and that customers on auto-pay exhibit 35–40% lower churn rates than manual payers.

5.2. Ingestion Pipeline

The ingestion pipeline is implemented in `rag/ingest.py` and is executed once prior to deployment. It performs the following steps:

1. **Loading:** The knowledge base markdown file is loaded using LangChain's `TextLoader`.
2. **Chunking:** The document is split into chunks of 400 characters with an 80-character overlap using `RecursiveCharacterTextSplitter`. The overlap preserves context across chunk boundaries, preventing strategy descriptions from being severed mid-sentence.
3. **Embedding:** Each chunk is embedded using the `all-MiniLM-L6-v2` Sentence Transformer model via `HuggingFaceEmbeddings`. This model produces 384-dimensional dense vectors and runs entirely on local CPU — no external embedding API is required.
4. **Persistence:** The embedded chunks are written to a ChromaDB persistent vector store at `rag/chroma_db/`. The store is pre-built and committed to the repository, ensuring zero-latency RAG on application startup.

5.3. Retrieval Strategy

At query time, `rag/retriever.py` loads the persisted ChromaDB store and performs Max Marginal Relevance (MMR) search with parameters $k = 3$ and `fetch_k` $= 9$, and a diversity parameter $\lambda = 0.5$. MMR balances two objectives: relevance to the query (cosine similarity to the query embedding) and diversity relative to already-selected chunks (maximum marginal relevance). This prevents the retriever from returning three near-duplicate chunks that describe the same strategy in slightly different wording, which would waste the LLM's context window and produce repetitive recommendations.

If the ChromaDB store is absent on startup (e.g., first local run after cloning), the retriever automatically triggers `build_vector_store()` to rebuild it before serving the first query.

6. LLM and Prompting Strategy

6.1. Model Selection

The system uses Groq's `llama-3.3-70b-versatile` model, accessed via the `langchain-groq` integration. This model was selected for the following reasons:

- **Free tier availability:** Groq provides a generous free-tier API with no credit card requirement.
- **Inference speed:** Groq's LPU hardware delivers significantly lower latency than equivalent GPU-based inference, which is important for the user-facing web application.
- **Model quality:** Llama 3.3 70B produces high-quality, instruction-following outputs suitable for structured business communications.

The LLM is instantiated with `temperature=0.3` to balance coherence with modest output variety across different customer profiles.

6.2. Prompt Design and Hallucination Mitigation

Three distinct prompt templates are used across nodes 3–5, each designed to constrain the LLM's output format and reduce hallucination:

- **EXPLANATION_PROMPT:** Instructs the LLM to produce one paragraph only, referencing specific customer feature values (e.g., tenure of 2 months, monthly charge of $90). By anchoring the explanation to concrete values from the state, the prompt prevents the model from generating generic churn narratives unrelated to the actual customer.
- **RECOMMENDATION_PROMPT:** Instructs the LLM to produce a numbered list of exactly three items, each 1–2 sentences, referencing the customer's specific situation. The retrieved retention context is provided as authoritative source material, grounding recommendations in domain knowledge rather than LLM parametric memory.
- **SUMMARY_PROMPT:** Constrains the output to exactly one sentence under 25 words, preventing verbosity in the executive summary field.

The inclusion of retrieved context in all three prompts is the primary hallucination mitigation strategy. Rather than asking the LLM to generate retention advice from general knowledge, the prompts explicitly frame the retrieved chunks as the authoritative source and instruct the model to ground its outputs in that material.

7. Results and Qualitative Analysis

7.1. Milestone 1 Model Performance (Retained)

The Logistic Regression model from Milestone 1 continues to serve as the prediction backbone of the agentic system. Its performance on the held-out test set is reproduced here for reference.

Table 1: Model performance on the held-out 20% test set (1,409 samples)

Model	Accuracy	Precision	F1-Score	ROC-AUC
Logistic Regression	**0.8070**	**0.6584**	**0.6092**	**0.8416**
Random Forest	0.7864	0.6237	0.5501	0.8251
XGBoost	0.7850	0.6079	0.5690	0.8214
Decision Tree	0.7559	0.5387	0.5486	0.7607

7.2. Qualitative Example: High-Risk Customer

To illustrate the agent's end-to-end reasoning capability, Table 2 presents the input profile for a representative high-risk customer, followed by the agent's structured output.

Table 2: Example high-risk customer input profile

Attribute	Value	Attribute	Value
Contract	Month-to-month	Tech Support	No
Tenure	2 months	Online Security	No
Monthly Charge	$90.00	Internet	Fiber optic
Payment Method	Electronic check	Streaming TV	Yes
Senior Citizen	No	Partner	No

Agent Output:

- **Churn Probability:** 78% **Risk Level:** High
- **Risk Factors:** month-to-month contract; new customer (short tenure); fiber optic subscriber (premium pricing)
- **Explanation:** *This customer is at high risk of churning primarily due to a combination of low commitment and high cost exposure. With only two months of tenure and a month-to-month contract, they have experienced minimal lock-in and face no financial penalty for leaving. Their $90 monthly charge on a fiber optic plan places them in a price-sensitive segment where competitors frequently target with promotional rates. The absence of tech support and online security further reduces their perceived value of the service.*
- **Recommendation 1:** Offer a 15–20% monthly discount in exchange for upgrading to a one-year contract, framing this as a loyalty reward rather than a discount to increase perceived value and reduce the psychological barrier to commitment.
- **Recommendation 2:** Enrol the customer in auto-pay to receive a $5/month bill credit, converting the electronic check payment — a known disengagement indicator — into an automatic interaction that reduces monthly decision points.

- **Recommendation 3:** Offer a free three-month trial of tech support and online security, which data shows converts at 45–55% and significantly reduces churn in fiber optic subscribers by increasing perceived service value.
- **Executive Summary:** *Prioritise immediate contract upgrade offer with loyalty discount for this high-cost, low-tenure fiber optic customer.*

7.3. Qualitative Example: Low-Risk Customer

For contrast, the same pipeline applied to a loyal customer profile (two-year contract, 60 months tenure, $55/month, tech support enabled) returns a churn probability of approximately 9%, risk level Low, and recommendations focused on proactive satisfaction outreach and loyalty programme enrolment rather than urgent intervention — demonstrating that the agent correctly calibrates its outputs to the customer's actual risk profile.

8. Deployment and Technology Stack

8.1. Gradio Application

The Gradio application (`app.py`) provides two tabs. The **AI Agent Analysis** tab is the primary Milestone 2 interface, exposing the full LangGraph pipeline. The **Quick Predict** tab retains the Milestone 1 ML predictor for fast, offline churn probability estimation with no API key required. Both tabs share the same three-tab input layout (Demographics, Account & Billing, Services).

The Agent Analysis tab renders a structured HTML report comprising the churn probability score, risk level badge, risk factor tags, the LLM explanation, numbered recommendations, and an executive summary. A collapsible "Retrieved Knowledge (RAG)" accordion displays the raw source chunks used by the agent, providing full transparency into the retrieval process.

8.2. Technology Stack

Table 3: Technology stack for Milestone 2

Component	Technology
ML Model	Logistic Regression (`scikit-learn`)
Agent Framework	LangGraph (`StateGraph`, 5 nodes)
LLM	Groq `llama-3.3-70b-versatile` via `langchain-groq`
Vector Store	ChromaDB (`langchain-chroma`)
Embeddings	`all-MiniLM-L6-v2` (Sentence Transformers, local)
RAG Framework	LangChain (`TextLoader`, `RecursiveCharacterTextSplitter`)
Web UI	Gradio
Hosting	Hugging Face Spaces
API Key Management	HF Secrets (`GROQ_API_KEY`)

9. Discussion

9.1. Strengths of the Agentic Approach

The five-node LangGraph architecture provides several advantages over a single-prompt approach. First, explicit state management ensures that each node receives precisely the context it needs without over-populating the LLM's context window. Second, the separation of explanation and recommendation into distinct nodes — each with its own system prompt and persona — produces higher-quality outputs than a single monolithic prompt attempting to do both. Third, the singleton agent pattern and lazy RAG initialisation ensure that the deployed application starts quickly and serves subsequent requests with minimal overhead.

9.2. RAG vs. Pure LLM Generation

The choice to use RAG rather than relying on the LLM's parametric knowledge for retention strategy generation was motivated by two considerations. First, retention strategies are domain-specific and highly context-dependent — a generic LLM may produce plausible-sounding but strategically unsound recommendations. Second, the retrieved context provides an auditable source for every recommendation, which is important in a business-facing application where recommendations must be defensible. The RAG accordion in the UI exposes these sources directly to the end user.

9.3. Limitations

1. **LLM API dependency:** The AI Agent Analysis tab requires a valid `GROQ_API_KEY`. If the key is absent or the Groq service is unavailable, the agent tab degrades gracefully but cannot produce recommendations.
2. **Static knowledge base:** The retention strategies knowledge base is a fixed markdown document. In a production deployment, this would ideally be a live, continuously updated database of retention outcomes.
3. **No feedback loop:** The system does not incorporate any mechanism for learning from outcomes — for example, tracking which recommendations led to successful retention and updating the knowledge base accordingly.
4. **Inherited ML limitations:** The underlying Logistic Regression model still suffers from the class imbalance limitation documented in Milestone 1 (recall of 0.57 on the Churn class). This affects the accuracy of the probability estimate passed to the agent.

10. Conclusion

This report has documented the end-to-end design and implementation of an agentic AI retention strategy assistant built on the churn prediction foundation established in Milestone 1. The system successfully integrates a five-node LangGraph state machine, a ChromaDB RAG pipeline with local sentence transformer embeddings, and a free-tier Groq LLM into a publicly deployed, professionally designed web application.

The agentic approach demonstrates that prediction and intervention can be combined into a single autonomous pipeline. Given a customer profile, the system autonomously runs the ML model, retrieves domain-specific retention knowledge, generates a grounded explanation, and produces actionable recommendations — reducing the cognitive load on human retention teams and enabling faster, more consistent intervention decisions.

The project achieves all Milestone 2 objectives: LangGraph workflow with explicit state management, correct RAG implementation with MMR retrieval, structured output clarity, graceful error handling, ethical disclaimers through source attribution, and successful deployment on a free-tier public hosting platform.

11. Team Contributions

All four team members contributed equally to the overall project. The division of responsibilities across the Milestone 2 deliverables was as follows.

Hemanth Tenneti (2401010186) led the technical implementation of the agentic system. He was responsible for the complete LangGraph state machine (`churn_agent.py`), including the `AgentState` schema, all five node functions, the error-safe wrapper pattern, the singleton agent interface, and the graph compilation. He also implemented the LangChain tool wrappers in `tools.py` and the feature encoding pipeline used by the prediction tool.

Aaloke Eppalapalli (2401010005) was responsible for the Gradio application and deployment. He redesigned `app.py` to support the two-tab architecture, implemented

the AI Agent Analysis output panel with structured HTML rendering including risk factor tags, recommendation cards, and the executive summary section, and managed the Hugging Face Spaces deployment including the GROQ_API_KEY secret configuration.

Arina Ali (2401020114) was responsible for the project report. She authored this document in LaTeX, covering the agentic workflow, RAG pipeline, qualitative examples, and system discussion. She also designed the LLM prompt templates in `prompts.py`, including the hallucination mitigation strategies embedded in the system instructions for all three prompt templates.

Shrihari K N (2401010448) was responsible for the RAG pipeline and project video. He implemented the ingestion pipeline (`rag/ingest.py`) and the retrieval interface (`rag/retriever.py`) curated and structured the retention strategies knowledge base (`retention_strategies.md`), and configured the ChromaDB persistent vector store. He also produced and edited the five-minute end-semester demonstration video.

References

[1] Reichheld, F. F. and Sasser, W. E. (1990). *Zero defections: Quality comes to services.* Harvard Business Review, 68(5), 105–111.

[2] IBM. (2018). *Telco Customer Churn* [Dataset]. Kaggle. Retrieved from https://www.kaggle.com/datasets/blastchar/telco-customer-churn

[3] Hastie, T., Tibshirani, R., and Friedman, J. (2009). *The Elements of Statistical Learning: Data Mining, Inference, and Prediction* (2nd ed.). Springer.

[4] LangChain AI. (2024). *LangGraph: Build stateful, multi-actor applications with LLMs.* Retrieved from https://langchain-ai.github.io/langgraph/

[5] Chroma. (2024). *Chroma: The AI-native open-source embedding database.* Retrieved from https://www.trychroma.com/

[6] Reimers, N. and Gurevych, I. (2019). *Sentence-BERT: Sentence embeddings using Siamese BERT-networks.* Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing (EMNLP).

[7] Lewis, P. et al. (2020). *Retrieval-Augmented Generation for knowledge-intensive NLP tasks.* Advances in Neural Information Processing Systems (NeurIPS), 33, 9459–9474.

[8] Pedregosa, F. et al. (2011). *Scikit-learn: Machine learning in Python.* Journal of Machine Learning Research, 12, 2825–2830.